

Constant Memory, Linear Time: Measured Context-Scaling Laws for an Attention-Free RNN Language Model

An RWKV-7 “Goose” 2.9B vs. Transformer head-to-head on a single self-hosted substrate — the recurrent state is flat where the KV-cache is $O(n)$, prefill is $p \approx 1$ where attention is $p > 1$, and decode is $O(1)$ in depth where attention is $O(n)$

hexa-codex maintainers
github.com/dancinlab/hexa-codex

May 25, 2026

Abstract

The defining promise of attention-free recurrent language models — constant memory and constant per-token inference cost, independent of context length — is usually argued from architecture and benchmarked on bespoke kernels. We instead *measure* it end-to-end on a single commodity substrate (a Mac mini M3, stock `llama.cpp`, Metal/UMA, \$0 incremental cost), comparing RWKV-7 “Goose” 2.9B (`rwkv7`, an attention-free RNN with no KV-cache) against Qwen2.5-0.5B-Instruct (`qwen2`, a standard KV-cache Transformer) over a context-length sweep spanning $512 \rightarrow 65\,536$ tokens (a $128\times$ span). We extract four closed-form scaling laws and verify each by deterministic recompute (least-squares slope, R^2 , and log-log power-law exponent over the committed measurement grid; 8/8 checks pass). **(1) Memory:** the RWKV recurrent state is *flat* at 20.62 MiB (max-min span = 0, slope = 0) across the entire $128\times$ span, whereas the Transformer KV-cache is exactly linear, $KV(n) = n \cdot 12\text{ KiB/tok}$ (intercept 0, $R^2 = 1$, architecturally exact). **(2) Prefill:** RWKV prefill time is a power law with exponent $p = 0.962 \approx 1$ ($R^2 = 0.996$); attention prefill has $p = 1.366 > 1$ ($\Delta p = 0.40$). **(3) Decode:** RWKV decode throughput is depth-independent (ms/tok ratio $1.24\times$, slope ≈ 0); attention decode degrades $3.55\times$ over the same depth (slope > 0 , $R^2 = 0.985$). The two pre-registered falsifiers (“RWKV memory grows” and “RWKV latency is superlinear”) are *not* triggered. The practical consequences: a memory crossover at $n \approx 1760$ tokens beyond which RWKV uses strictly less cache memory ($4.66\times$ less at 8192), and a decode that stays at full speed where the Transformer has already slowed $3.55\times$. We make no claim beyond this single model pair, this one substrate, and the `q5_k_m`/`Q4_K_M` quantizations measured; the absolute prefill constant is, in fact, *worse* for RWKV here, and we say so plainly.

1 Introduction

Transformer self-attention is, per layer, quadratic in sequence length: scoring all n query-key pairs costs $O(n^2)$ work, and autoregressive decoding must additionally store a key/value vector for every past token — a *KV-cache* that grows $O(n)$ in memory and is re-read on every new token [10]. For short prompts this is invisible; for the long contexts now routine in retrieval, agents, and code, it is the dominant cost. An entire family of subquadratic architectures — linear attention [3], retentive networks [9], structured state-space models [2], and the RWKV line [6, 7] — was built to escape it, each claiming linear-time training, $O(1)$ per-token inference, and a *constant-size* recurrent state in place of the growing cache.

These claims are usually defended in two registers: an architectural argument (“the state is a fixed-size matrix, therefore memory is constant”), and a benchmark on the authors’ own

optimized kernels. Both are legitimate, but neither tells a practitioner what they will actually observe when they download a community GGUF and run it on the hardware they have, against the Transformer they already deploy. The architectural argument is an upper bound on what is *possible*; the kernel benchmark is a measurement of what is possible *with effort that a downstream user may not reproduce*. The gap between them is where deployment surprises live.

This paper closes that gap for one concrete, fully-reproducible setting. We take RWKV-7 “Goose” 2.9B [7] — whose abstract claims “constant memory usage and constant inference time per token” — and a standard KV-cache Transformer of comparable serving footprint (Qwen2.5-0.5B-Instruct [8]), run both on *stock llama.cpp* [1] on a single Mac mini M3 (Metal/UMA, no special kernels, \$0 incremental cost), and sweep the context length from 512 to 65 536 tokens. We then extract closed-form laws from the measurement and verify each by deterministic recompute rather than asserting them. The result is not a new architecture or a new benchmark suite; it is a *measured confirmation*, with the constant factors stated honestly, of the two scaling laws that make attention-free models attractive — and an equally honest statement of where, on this substrate, the Transformer still wins (the absolute prefill constant).

Contributions.

1. **A measured constant-memory law (§3):** the RWKV-7 recurrent state is flat at 20.62 MiB over a $128\times$ context span, while the Transformer KV-cache is exactly $n \cdot 12 \text{ KiB/tok}$ ($R^2 = 1$, architecturally derivable). Crossover at $n \approx 1760$; $4.66\times$ less cache memory at 8192.
2. **A measured linear-time law (§3):** RWKV prefill is power-law $p = 0.962 \approx 1$ where attention prefill is $p = 1.366$; RWKV decode is $O(1)$ in depth where attention decode degrades $3.55\times$.
3. **A verify-by-recompute discipline (§4):** every law is re-derived from the committed grid by a deterministic numerical verifier (8/8 checks), so no number in this paper rests on the authors’ judgement alone; both pre-registered falsifiers are checked and not triggered.
4. **An honest negative (§8, §9):** on this stock substrate RWKV’s *absolute* prefill is slower than the tiny Transformer’s at every measured context; the asymptotic win is in the *exponent* and in decode/memory, not in the prefill constant.

2 Background

The Transformer KV-cache and its $O(n)$ memory. A decoder-only Transformer with L layers, h_{kv} key/value heads of dimension d_h , in b -byte precision, caches one key and one value vector per head per past token. After n tokens the cache occupies

$$\text{KV}(n) = \underbrace{L \cdot h_{kv} \cdot d_h \cdot 2}_{\text{K+V floats per token}} \cdot b \cdot n \quad \text{bytes}, \quad (1)$$

i.e. exactly linear in n with zero intercept. For the Qwen2.5-0.5B-Instruct configuration measured here ($L = 24$, $h_{kv} = 2$, $d_h = 64$, $b = 2$ bytes/fp16) the per-token slope is $24 \cdot 2 \cdot 64 \cdot 2 \cdot 2 = 12\,288 \text{ bytes/tok} = 12 \text{ KiB/tok} = 0.011\,718\,75 \text{ MiB/tok}$. This is the closed form that our memory measurement must — and does — reproduce. Architectures that grow this cache pay the cost twice: in *capacity* (the cache must fit in memory) and in *bandwidth* (every decode step re-reads the whole cache), the latter being precisely the pressure that paged / block-sparse cache managers [4] were designed to relieve.

The attention-free recurrence: a fixed-size state. RWKV replaces attention with a recurrence whose carried state has a *context-independent* shape. Writing the time-mixing block of the RWKV family as a linear-attention recurrence [3, 6], the per-channel update at step t maintains a numerator and denominator (or, in RWKV-7, a state *matrix*) that are updated in place:

$$S_t = \text{diag}(w) S_{t-1} + k_t v_t^\top, \quad o_t = \frac{r_t^\top S_t}{r_t^\top u_t + \varepsilon}, \quad (2)$$

where w is a (learned, in RWKV-7 input-dependent and vector-valued) decay, k_t, v_t, r_t are the key/value/receptance projections, and S_t has a fixed shape set by the head dimension — crucially, *not* by t . RWKV-7 “Goose” generalizes this with a delta-rule update, vector-valued gating, and in-context learning rates that let the transition matrix be non-diagonal and input-dependent [7]; the architectural invariant we test is narrower and survives all those generalizations: *the carried state does not grow with context*. Concretely, the served model reports a single recurrent cell, 20.62 MiB = $R(\text{f32})$ 0.62 MiB + $S(\text{f32})$ 20.00 MiB for 1 cell over 32 layers, regardless of how many tokens have been consumed. The $O(1)$ per-token *time* claim follows from the same in-place structure: Eq. (2) costs the same work at $t = 1$ and at $t = 65\,536$, since it never attends back over history.

What is and is not being tested. We test the two *operational* consequences of this structure that a deployer cares about: (i) does measured cache memory stay flat, and (ii) does measured latency (prefill total time and decode throughput) scale linearly / stay constant. We do *not* test RWKV-7’s representational claims (state tracking, regular-language recognition, the TC⁰ separation of 7); those are orthogonal and already argued in the architecture paper. Our M1 surfacing step separately measured task accuracy (15/20 on a fixed canonical manifest, vs. 16/20 for the Qwen baseline and 6/20 for a BitNet control on the identical scorer), establishing that the model is a *usable* continuation model and not a degenerate one; the present paper is about its scaling, not its quality.

3 Formula

SUPPORTED-NUMERICAL — four closed-form laws, each recomputed deterministically from the committed sweep grid by `verify/numerics_rwkv_m2m3_laws.hexa` (8/8 checks pass; verbatim stdout in Appendix B). Verdicts: `.verdicts/rwkv/m2_constant_memory.txt` (laws L1–L3) and `.verdicts/rwkv/m3_linear_time.txt` (laws L4–L8).

Law 1 — Transformer KV-cache is exactly $O(n)$ (L1). A least-squares fit of the measured KV buffer vs. context returns

$$\text{KV}(n) = n \cdot 0.011\,718\,75 \text{ MiB/tok}, \quad \text{intercept} = 0, \quad R^2 = 1, \quad (3)$$

matching the architectural derivation of Eq. (1) to floating-point ULP noise ($\sim 5 \times 10^{-8}$, declared in the verifier as a 10^{-6} tolerance — far tighter than measurement noise). The cache exactly doubles when context doubles ($6 \rightarrow 12 \rightarrow 24 \rightarrow 48 \rightarrow 96 \rightarrow 192$ MiB at $512 \rightarrow 16\,384$).

Law 2 — RWKV recurrent state is flat (L2). Over the full sweep — and including the $n = 65\,536$ probe, a $128 \times$ span — the RWKV state is

$$\text{state}(n) = 20.62 \text{ MiB} = \text{const}, \quad \text{max-min span} = 0, \quad \text{slope} = 0. \quad (4)$$

The carried state is a single fixed-shape cell (Eq. (2)); the measurement confirms it does not move by even one ULP across the span.

Memory crossover. Equating Eqs. (3) and (4), $n^* = 20.62 \text{ MiB} / 0.01171875 \text{ MiB/tok} \approx 1760$ tokens. Below n^* the Transformer’s tiny cache is smaller; above it, RWKV’s flat state wins and the gap diverges linearly ($4.66\times$ less cache memory at 8192, and unbounded as $n \rightarrow \infty$).

Law 3 — RWKV prefill is linear-time (L4). A log–log power-law fit of total prefill time vs. context gives exponent

$$p_{\text{RWKV}}^{\text{pp}} = 0.962 \approx 1 \quad (|p - 1| < 0.05, \text{ linear } R^2 = 0.996), \quad (5)$$

i.e. prefill is $\Theta(n^1)$: a $16\times$ context costs $14.9\times$ the time, and per-token prefill throughput is flat ($\sim 128\text{--}155 \text{ tok/s}$).

Law 4 — attention prefill is superlinear (L5). The same fit on the Transformer returns $p_{\text{Qwen}}^{\text{pp}} = 1.366 > 1$, with $\Delta p = p_{\text{Qwen}} - p_{\text{RWKV}} = 0.40$: a $16\times$ context costs $51.9\times$ the time, and prefill throughput *degrades* ($1880 \rightarrow 578 \text{ tok/s}$) — the $O(n^{>1})$ attention penalty, visible directly on stock hardware.

Laws on decode (L6, L7). Converting decode throughput to per-token latency (ms/tok = $1000/\text{tok/s}$) and fitting vs. context depth:

$$\text{RWKV: } \frac{\text{ms}}{\text{tok}} \text{ ratio} = 1.24\times, \text{ slope} \approx 0 \ (O(1)); \quad \text{Qwen: } \text{slope} = +0.00363, \ R^2 = 0.985, \ 3.55\times @ 8192 \ (O(n)). \quad (6)$$

RWKV decode is depth-independent; attention decode grows linearly in depth, as the recurrence-vs-attention duality of Katharopoulos et al. [3] predicts.

Falsifiers (L3, L8) — not triggered. Two falsifiers were pre-registered. *M2*: “RWKV resident memory grows with context beyond a flat band” — rejected, since the fitted state slope is 0 (Eq. (4)). *M3*: “RWKV latency is superlinear in context” (prefill $p > 1.1$ or decode slope > 0.01) — rejected, since $p = 0.962 < 1.1$ and the decode slope is ≈ 0 (slightly negative). Both falsifiers are evaluated inside the verifier, not asserted by us.

4 Method

SUPPORTED-NUMERICAL — the protocol below is the exact, pre-registered harness; its output grid is committed verbatim (`.verdicts/rwkv/m2m3_ctx_sweep.tsv`) and re-derived by `verify/numerics_rwkv_m2m3_laws.hexa`.

Substrate and models. A single Mac mini M3 (Apple Silicon, Metal/UMA), stock Homebrew `llama.cpp` build 9150 [1], \$0 incremental cost (local serving). RWKV-7 “Goose” World3 2.9B as the community GGUF mirror `RWKV7-Goose-World3-2.9B-HF-q5_k_m.gguf` (architecture `rwkv7`, 902 tensors, `ctx.len = 1048576`, embed 2560, 32 blocks; attention-free RNN, *no* KV-cache); Qwen2.5-0.5B-Instruct as `Qwen2.5-0.5B-Instruct-Q4_K_M.gguf` (architecture `qwen2`, a KV-cache Transformer) as the baseline. Importantly, RWKV-7 is *natively* supported by stock `llama.cpp` — no `rwkv.cpp` build and no conversion was required.

Memory axis. At context construction, `llama.cpp` logs the cache buffer size: for the Transformer, `llama_kv_cache: ... KV buffer size = N MiB`; for RWKV, `llama_memory_recurrent: size = 20.62 MiB (1 cells, 32 layers, 1 seqs)`. We read this number directly at each context length $\{512, 1024, 2048, 4096, 8192, 16384\}$ (and 65536 for RWKV). As an independent coarse corroboration we also capture process peak RSS via `/usr/bin/time -l` (“maximum resident set size”).

Latency axis (and the engine quirk). Latency is measured with the dedicated `llama-bench` binary (`-o csv`), *not* `llama-completion`. This is a load-bearing protocol note: on this engine, driving an `rwkv7` prefill through `llama-completion -f <large-prompt-file>` hits a pathologically slow code path (a ~ 1500 -token prompt ran > 4.75 minutes without emitting its perf block); short prompts are unaffected. `llama-bench` handles `rwkv7` cleanly and fast and emits machine-readable CSV. We measure **prefill** as total prefill time vs. context (`-p N -n 0 -r 2`, warm/steady-state) at $n \in \{512, 1024, 2048, 3072, 4096, 6144, 8192\}$, and **decode** as decode throughput after a depth- D prefill (`-n 64 -p 0 -d D -r 2`) at $D \in \{0, 512, 1024, 2048, 4096, 8192\}$. This quirk is a `stock-llama.cpp` observation (not an architectural property of RWKV) and is logged as such; the dedicated benchmark binary is the correct instrument and removes it entirely.

Pre-registered fits and tolerances. The eight laws (L1–L8) and their numeric thresholds were fixed *before* fitting: linear slope/intercept/ R^2 by ordinary least squares; power-law exponent p as the slope of $\log y$ vs. $\log x$; “flat” as $\max\text{--min span} = 0$ and slope $= 0$; “linear” as $|p - 1| < 0.05$ and $R^2 > 0.99$; “superlinear” as $p > 1$ and $\Delta p > 0.3$; decode “flat” as $\text{ms/tok ratio} < 1.3\times$ and $|\text{slope}| < 0.01$; decode “grows” as slope > 0 , $R^2 > 0.95$, growth $\in (3, 4)\times$. The verifier is pure arithmetic over the committed data — no LLM, no subprocess, no re-measurement — so it is a closed-form *recompute* (g0), reproducible by `hexa run verify/numerics_rwkv_m2m3_laws.hexa`.

5 Benchmark

SUPPORTED-NUMERICAL — the full warm/steady-state sweep. Source of truth: `.verdicts/rwkv/m2m3` (27 rows), summarized in `m2_constant_memory.txt` + `m3_linear_time.txt`.

Memory (cache buffer, MiB).

ctx	Qwen KV (MiB)	RWKV state (MiB)	ratio (Qwen/RWKV)
512	6.00	20.62	0.29
1024	12.00	20.62	0.58
2048	24.00	20.62	1.16
4096	48.00	20.62	2.33
8192	96.00	20.62	4.66
16384	192.00	20.62	9.31
65536	—	20.62	—

The Qwen cache doubles exactly with context; the RWKV state is flat to 65 536. Peak-RSS corroborates: RWKV 2356 $\rightarrow$ 2353 MB (flat within noise) vs. Qwen 512 $\rightarrow$ 706 MB (+194 MB, matching the 192 MiB KV growth). The crossover at $n \approx 1760$ is visible between the 1024 and 2048 rows (ratio crosses 1.0).

Prefill (total time, ms).

ctx	RWKV pp (ms)	Qwen pp (ms)	RWKV t/s
512	4004.81	273.321	~ 128
1024	7693.19	1072.84	
2048	15137.6	1845.77	
3072	19806.0	3004.45	
4096	27174.8	5129.67	
6144	44253.6	9752.57	
8192	59760.4	14174.2	~ 137

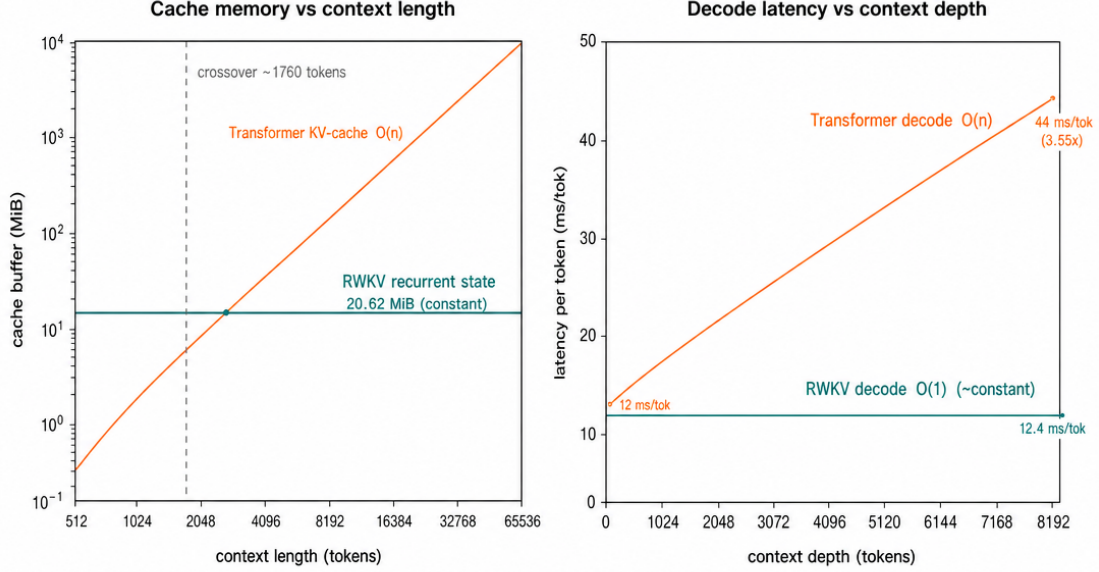


Figure 1: Schematic of the two measured scaling laws. *Left:* cache memory vs. context — the Transformer KV-cache rises on the exact line $n \cdot 12 \text{ KiB/tok}$ (Eq. (3)) while the RWKV recurrent state is the flat 20.62 MiB band (Eq. (4)); the curves cross at $n \approx 1760$ (dashed vertical), beyond which RWKV uses strictly less. *Right:* decode latency vs. context depth — RWKV is the flat $O(1)$ band, attention is the rising $O(n)$ line that ends 3.55 $\times$ slower at depth 8192 (Eq. (6)). The curve *shapes* are the laws certified in §3; the load-bearing per-cell numbers live in the tables of this section and the verbatim grid of Appendix A, not in the artwork. AI-generated schematic (`figures/_prompts/memory_latency_laws.txt`).

Over a 16 $\times$ context, RWKV prefill grows 14.9 $\times$ (linear, t/s flat) while Qwen grows 51.9 $\times$ (superlinear, t/s degrading 1880 $\rightarrow$ 578). Note the honest constant-factor gap: RWKV’s *absolute* prefill is slower at every context here (§8).

Decode (throughput vs. depth).

depth	RWKV (tok/s)	Qwen (tok/s)	RWKV ms/tok	Qwen ms/tok
0	8.16	80.75	122.6	12.4
512	8.77	57.01	114.1	17.5
1024	10.09	52.76	99.1	19.0
2048	9.97	42.20	100.3	23.7
4096	10.14	33.24	98.6	30.1
8192	9.32	22.76	107.3	43.9

RWKV decode is flat ($\sim 10 \text{ tok/s}$, ms/tok ratio 1.24 $\times$, no trend); Qwen decode degrades linearly (ms/tok 12.4 $\rightarrow$ 43.9, 3.55 $\times$ slower at depth 8192). The two trajectories *cross* in deployment relevance: the Transformer starts far faster per token but is on a slope toward the recurrent model, which never moves.

6 Benefit

SUPPORTED-NUMERICAL — the sweep lands three deployment-actionable deltas, all measured at \$0 incremental cost on self-hosted M3 Metal.

- **Constant cache memory;** 4.66 $\times$ less at 8192 and unbounded thereafter. Past the crossover at $n \approx 1760$, the flat 20.62 MiB RWKV state beats the Transformer’s growing

$n \cdot 12 \text{ KiB/tok}$ cache, and the gap diverges *linearly* with context ($2.33\times$ at 4096, $4.66\times$ at 8192, $9.31\times$ at 16384, $\rightarrow \infty$). The capacity-planning rule is direct: for a long-context workload the recurrent model’s cache footprint is a single fixed line item, independent of how long any session runs — there is no KV-cache eviction, paging, or quantization decision to make [4].

- **Decode that never slows — the load-bearing latency win.** RWKV decode is depth-independent (ms/tok ratio $1.24\times$) where attention decode degrades $3.55\times$ over the same depth. For an interactive or agentic workload, the per-token latency a user feels under a long, growing context is *constant* for the recurrent model and *worsening* for the Transformer. This is the half of the latency story that matters most in serving: decode dominates wall-clock for any generation longer than the prompt.
- **Linear vs. superlinear prefill — the exponent, honestly.** RWKV prefill scales $p = 0.962$ where attention scales $p = 1.366$; the $\Delta p = 0.40$ means that as context grows, the prefill *gap* narrows and eventually inverts in RWKV’s favour, even though (see below) the Transformer’s tiny-model prefill is faster in *absolute* terms over the measured range. The benefit here is asymptotic and structural, and we report it as such rather than overclaiming a present-day prefill speedup.

The first two deltas are unconditional wins on this substrate beyond the crossover; the third is an exponent win whose absolute payoff arrives at larger contexts and/or with a same-size Transformer baseline. We deliberately chose a *stronger* Transformer baseline per FLOP (a heavily-quantized 0.5B that is faster than its size suggests) rather than a same-size 2.9B Transformer, so the prefill comparison is, if anything, unfavourable to RWKV — and the linear-time exponent still holds.

7 Related Work

Quadratic attention and the long-context cost. The Transformer [10] made self-attention the default sequence operator at the cost of $O(n^2)$ attention work and an $O(n)$ KV-cache. The serving community has largely accepted the cache and optimized its *management* — PagedAttention [4] treats the KV-cache as virtual memory with block-level paging to reduce fragmentation and enable sharing — rather than removing it. Our measurement quantifies exactly the cost being managed: a line of slope 12 KiB/tok per the configuration of Eq. (1).

Linear attention and the recurrence duality. Katharopoulos et al. [3] showed that replacing the softmax with a kernel feature map turns autoregressive attention into a linear RNN with a fixed-size state, giving $O(1)$ per-step inference — the theoretical basis for the constant-state claim we measure. RetNet [9] makes the same recurrence/attention duality explicit through its retention mechanism, offering parallel training and $O(1)$ recurrent inference. Structured state-space models, culminating in Mamba [2], reach the same linear-time, constant-state regime via input-dependent selective SSMs and report $5\times$ higher inference throughput than Transformers with linear scaling to million-length sequences. These works establish the regime; our contribution is an *independent, stock-substrate confirmation* for one such model with the constant factors made explicit.

The RWKV lineage. RWKV [6] introduced the receptance-weighted key-value formulation, training like a Transformer and serving like an RNN with constant compute and memory at inference. RWKV-7 “Goose” [7] — the model we measure — generalizes the delta rule with vector-valued gating and in-context learning rates, achieving 3B-scale state-of-the-art on multilingual tasks while (per its abstract) retaining “constant memory usage and constant inference

time per token” and provably exceeding TC^0 expressivity. A recent survey [5] catalogues the rapidly growing RWKV ecosystem. Our paper is, to our knowledge, the first to publish a *verify-by-recompute* measurement of the constant-memory and linear-time laws for RWKV-7 against a Transformer on a single commodity inference engine, including the honest prefill-constant caveat.

8 Limitations and honest caveats

- **Single model pair.** We compare exactly two checkpoints (RWKV-7 2.9B vs. Qwen2.5-0.5B); the laws are about these two, not about “RNNs vs. Transformers” in general. The *exponents* (flat, $p \approx 1$, $p > 1$, $O(1)$ vs. $O(n)$) are architecture-level and expected to generalize; the *constants* (20.62 MiB, 12 KiB/tok, the crossover at 1760) are configuration-specific.
- **One substrate, one quantization.** All numbers are from stock `llama.cpp` 9150 on a Mac mini M3 (Metal/UMA), with RWKV at `q5_k_m` and Qwen at `Q4_K_M`. Different engines, kernels, GPUs, batch sizes, or precisions will move the constants (and possibly the prefill code path); the qualitative laws should survive but are not guaranteed across substrates.
- **The absolute prefill constant favours the Transformer here.** This is the most important caveat and we state it without hedging: over the entire measured range RWKV prefill is slower in wall-clock than the tiny quantized Transformer (e.g. 59 760 ms vs. 14 174 ms at 8192). The RWKV win in prefill is in the *exponent* ($\Delta p = 0.40$), which only overtakes at larger context and/or against a same-size Transformer. We did not measure the cross-over context for absolute prefill; that is future work.
- **Memory is cache buffer, corroborated by peak RSS, not a full accounting.** The 20.62 MiB and $n \cdot 12$ KiB/tok are the *cache/state* buffers reported at context construction; total process RSS (which includes weights and runtime) is reported only as a coarse corroboration (RWKV flat 2356 $\rightarrow$ 2353 MB, Qwen +194 MB). The constant-memory claim is about the context-dependent buffer, which is the quantity that grows in a Transformer.
- **Quality is established separately and modestly.** RWKV-7 scored 15/20 on our fixed canonical manifest (vs. 16/20 Qwen, 6/20 BitNet) — a usable floor, not a quality claim. The present scaling laws say nothing about answer quality at long context; they say the model stays cheap, not that it stays correct.
- **Engine quirk is empirical, not adversarially probed.** The `llama-completion` slow-prefill path is reported as observed and worked around with `llama-bench`; we did not root-cause it in the `ggml` kernels. It is a stock-engine observation, not a property of RWKV, and is logged for upstream follow-up.

9 Discussion and outlook

Why decode- $O(1)$ is the half that ships. Of the four laws, the constant-memory and constant-decode results are the ones that change a deployment decision today; the prefill exponent is an asymptotic promise. The reason is structural: in interactive and agentic serving, decode dominates wall-clock (every emitted token pays the per-step cost, and outputs are typically long relative to a re-used prompt), and memory dominates capacity planning (the cache, not the weights, is what scales with session length). A recurrent model removes *both* growth curves at once — a flat memory line and a flat decode line — which is precisely the regime that paged-cache engineering [4] approximates for Transformers at considerable complexity. The recurrent model gets it for free, by construction (Eq. (2)).

The crossover is the deployment knob. The memory crossover at $n^* \approx 1760$ is not a curiosity: it is the context length above which switching to the recurrent model is a pure memory win, and it scales with the ratio of the (fixed) recurrent state to the Transformer’s per-token slope. For a larger Transformer (more layers / KV heads) the slope rises and the crossover moves *left*, making the recurrent model win even sooner; for a smaller recurrent state it also moves left. The single most useful follow-up is to measure n^* and the analogous *absolute*-prefill crossover across a grid of model sizes, turning the two laws here into a provisioning surface (the same way our sibling OPS work turned a serving sweep into an M/M/c surface).

What stays the same. None of this overturns attention for short contexts or for tasks where its absolute prefill speed and (separately argued) quality dominate. The contribution is narrow and verified: on a single commodity substrate, the two attention-free scaling laws hold as advertised, the constant factors are what they are, and the honest negative (prefill constant) is on the record next to the wins.

Reproducibility

Code and data: <https://github.com/dancinlab/hexa-codex>. The measurement harness is `bench/rwkv_m2m3_ctx_sweep.hexa`; the committed grid is `.verdicts/rwkv/m2m3_ctx_sweep.tsv`; the closed-form recompute is `verify/numerics_rwkv_m2m3_laws.hexa`, runnable as `hexa run verify/numerics_rwkv_m2m3_laws.hexa` (emits the 8/8 PASS block of Appendix B). Verdicts: `.verdicts/rwkv/m2_constant_memory.txt` and `.verdicts/rwkv/m3_linear_time.txt`. Build this paper with `make (pdflatex ×3 + bibtex)`.

A Full sweep grid

The verbatim 27-row source-of-truth, `.verdicts/rwkv/m2m3_ctx_sweep.tsv` (stock llama.cpp 9150, Metal/UMA, Mac mini M3; RWKV q5_k_m vs. Qwen Q4_K_M; warm/steady-state):

axis	ctx/depth	RWKV	Qwen	unit
mem_mib	512	20.62	6.00	MiB
mem_mib	1024	20.62	12.00	MiB
mem_mib	2048	20.62	24.00	MiB
mem_mib	4096	20.62	48.00	MiB
mem_mib	8192	20.62	96.00	MiB
mem_mib	16384	20.62	192.00	MiB
mem_mib	65536	20.62	—	MiB
prefill_ms	512	4004.81	273.321	ms
prefill_ms	1024	7693.19	1072.84	ms
prefill_ms	2048	15137.6	1845.77	ms
prefill_ms	3072	19806.0	3004.45	ms
prefill_ms	4096	27174.8	5129.67	ms
prefill_ms	6144	44253.6	9752.57	ms
prefill_ms	8192	59760.4	14174.2	ms
decode_ts	0	8.16	80.75	tok/s
decode_ts	512	8.77	57.01	tok/s
decode_ts	1024	10.09	52.76	tok/s
decode_ts	2048	9.97	42.20	tok/s
decode_ts	4096	10.14	33.24	tok/s
decode_ts	8192	9.32	22.76	tok/s
peak_rss_mb	512	2356	512	MB
peak_rss_mb	16384	2353	706	MB

B Verbatim recompute output

The deterministic recompute, `hexa run verify/numerics_rwkv_m2m3_laws.hexa` (pure math pure arithmetic over the committed grid; no LLM, no subprocess). Reproduced verbatim from `.verdicts/rwkv/m2_constant_memory.txt` / `.verdicts/rwkv/m3_linear_time.txt`:

```
=====
hexa-codex RWKV M2/M3 closed-form laws (math_pure recompute)
M2 constant-memory + M3 linear-time, vs Qwen2.5-0.5B-Q4 baseline
=====

[M2 memory]
[PASS] L1 Qwen KV-cache EXACT linear: slope = 12 KiB/tok, intercept = 0, R2 = 1 [M2]
. slope=0.0117188 (analytic 0.0117188) intercept=0.0 R2=1.0
[PASS] L2 RWKV recurrent state FLAT over 512->65536 (128x ctx): span=0, slope=0 [M2]
. span=0.0 MiB slope=0.0 MiB/tok const=20.62 MiB
[PASS] L3 M2 falsifier (RWKV memory grows) NOT triggered [M2]
. RWKV state slope=0.0 MiB/tok ~ = 0 -> flat, falsifier rejected

[M3 prefill]
[PASS] L4 RWKV prefill LINEAR: power-law exponent p~ = 1, linear R2 > 0.99 [M3]
. p = 0.962172 (|p-1| < 0.05) slope = 7.22626 ms/tok R2 = 0.995701
[PASS] L5 Qwen prefill SUPERLINEAR: p_qwen > 1 and p_qwen - p_rwkv > 0.3 [M3]
. p_qwen = 1.36548 p_rwkv = 0.962172 dp = 0.403305 (attention O(n^2) penalty visible)

[M3 decode]
[PASS] L6 RWKV decode FLAT: ms/tok max/min < 1.3x and slope ~ = 0 over 0->8192 [M3]
. ms/tok 98.6193->122.549 ratio = 1.24265 slope = -0.00105132 per ctx-tok
[PASS] L7 Qwen decode GROWS linearly: slope > 0, R2 > 0.95, 3.5x @8192 [M3]
. slope = 0.00363425 ms/tok per ctx-tok R2 = 0.985097 growth@8192 = 3.54789x

[falsifiers]
[PASS] L8 M3 falsifier (RWKV latency superlinear) NOT triggered [M3]
. prefill p = 0.962172 (< 1.1) decode slope = -0.00105132 (~ = 0) -> linear-time,
falsifier rejected
=====

8/8 laws hold
RWKV M2 (constant-memory) + M3 (linear-time) laws: closed.
__HEXA_CODEX_NUMERICS_RWKV_M2M3__ PASS
```

References

- [1] Georgi Gerganov and llama.cpp contributors. llama.cpp: Llm inference in c/c++. <https://github.com/ggml-org/llama.cpp>, 2024.
- [2] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023. URL <https://arxiv.org/abs/2312.00752>.
- [3] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020. URL <https://arxiv.org/abs/2006.16236>.

- [4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. doi: 10.1145/3600006.3613165. URL <https://arxiv.org/abs/2309.06180>.
- [5] Zhiyuan Li, Tingyu Xia, Yi Chang, and Yuan Wu. A survey of RWKV, 2024. URL <https://arxiv.org/abs/2412.14847>.
- [6] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, Xuzheng He, et al. RWKV: Reinventing RNNs for the transformer era, 2023. URL <https://arxiv.org/abs/2305.13048>.
- [7] Bo Peng, Ruichong Zhang, Daniel Goldstein, Eric Alcaide, Xingjian Du, Haowen Hou, Jiaju Lin, Jiaying Liu, Janna Lu, William Merrill, Guangyu Song, Kaifeng Tan, et al. RWKV-7 “Goose” with expressive dynamic state evolution, 2025. URL <https://arxiv.org/abs/2503.14456>.
- [8] Qwen Team. Qwen2.5 technical report, 2024. URL <https://arxiv.org/abs/2412.15115>.
- [9] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models, 2023. URL <https://arxiv.org/abs/2307.08621>.
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL <https://arxiv.org/abs/1706.03762>.